Algorithm 3 Ullmann's Subgraph Matching Algorithm

1: **Input:** query graph $G_q = (V_q, E_q)$, data graph $G = (V, E)$
2: generate matching order $\pi = [u_1, u_2, \ldots, u_{|V_q|}]$ $(u_i \in V_q)$
3: ENUMERATE$(\emptyset, 1, \pi)$
4: **procedure** ENUMERATE(S, i, π)
5: $C_S(u_i) \leftarrow$ viable vertex candidates in G to match u_i
6: **for all** $v \in C_S(u_i)$ **do**
7: append S with (u_i, v)
8: **if** $|S| = k$ **then output** S
9: **else** ENUMERATE$(S, i+1, \pi)$
10: pop (u_i, v) from S

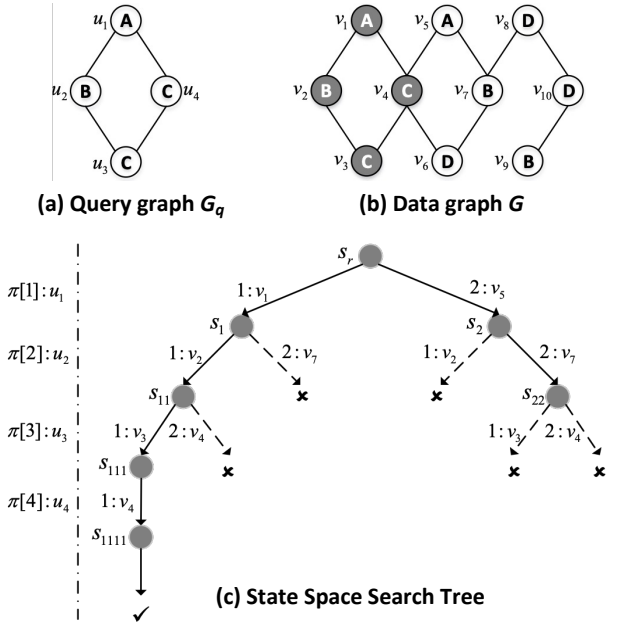


Fig. 11. Illustration of Subgraph Matching

Appendix A
Subgraph Matching

Subgraph matching (SM) is a representative SF problem that enumerates all subgraphs of a data graph G that are isomorphic to a query graph G_q. Many SM algorithms [3], [44] are based on Ullmann's backtracking framework [21], which orders query vertices and recursively extends a partial mapping while pruning infeasible candidates.

For completeness, we outline a basic Ullmann-style backtracking procedure in Algorithm 3. The algorithm incrementally matches data vertices to query vertices $u_1, \ldots, u_{|V_q|}$ according to a predefined matching order, maintaining the current partial mapping in S. At each recursion level, viable candidate vertices in G are derived for the next query vertex, and infeasible extensions are pruned early based on adjacency constraints. The search proceeds recursively until either a complete match is found or no further extension is possible.

This process explores a state-space tree as illustrated in Figure 11. A node at level i corresponds to a partial mapping

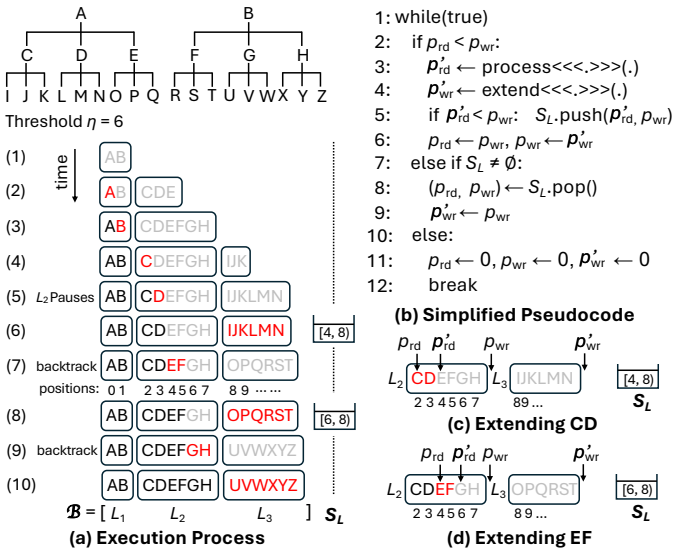


Fig. 12. Example of Hybrid BFS-DFS Subgraph Extension

that matches the query vertex sequence prefix $[u_1, \ldots, u_i]$. For example, the partial mapping $S = [(u_1, v_5), (u_2, v_7), (u_3, v_4)]$ is pruned due to the absence of an edge (v_7, v_4) corresponding to the query edge (u_2, u_3). Various optimizations adopted by modern SM algorithms [44]—such as improving the matching order (Line 2) and tightening candidate sets (Line 5)—aim to reduce the search space but preserve this fundamental backtracking structure.

Importantly, the state-space tree explored by Ullmann-style algorithms is also a subgraph enumeration tree over G: each node represents a subgraph in G that matches a prefix of the query graph.

Appendix B
GPU Worker Backtracking

Figure 12(b) presents the GPU worker's backtracking control loop for implementing the hybrid BFS-DFS extension described in Section III-A. Each iteration of the loop extends subgraphs from the current level L_i to the next level L_{i+1}.

For ease of presentation, let us continue to assume that each subgraph occupies only one logical position in the device buffer $\mathcal{B}$, and Appendix C will generalize it to the actual variable-length subgraph representation.

We represent L_i as a contiguous segment $\mathcal{B}[p_{rd}, p_{wr})$ in $\mathcal{B}$. Newly generated subgraphs of L_{i+1} are appended starting at p_{wr}, with p'_{wr} tracking the current end of the output segment. Meanwhile, p'_{rd} tracks the next parent in L_i to be processed: $\mathcal{B}[p_{rd}, p'_{rd})$ has been processed, while $\mathcal{B}[p'_{rd}, p_{wr})$ remains unprocessed. The evolution of these pointers is illustrated in Figure 12(c)–(d).

Since multiple GPU warps concurrently consume parents from L_i and append children to L_{i+1}, p'_{rd} and p'_{wr} act as shared read and write pointers, respectively, and are advanced atomically as warps acquire parents and reserve output positions.

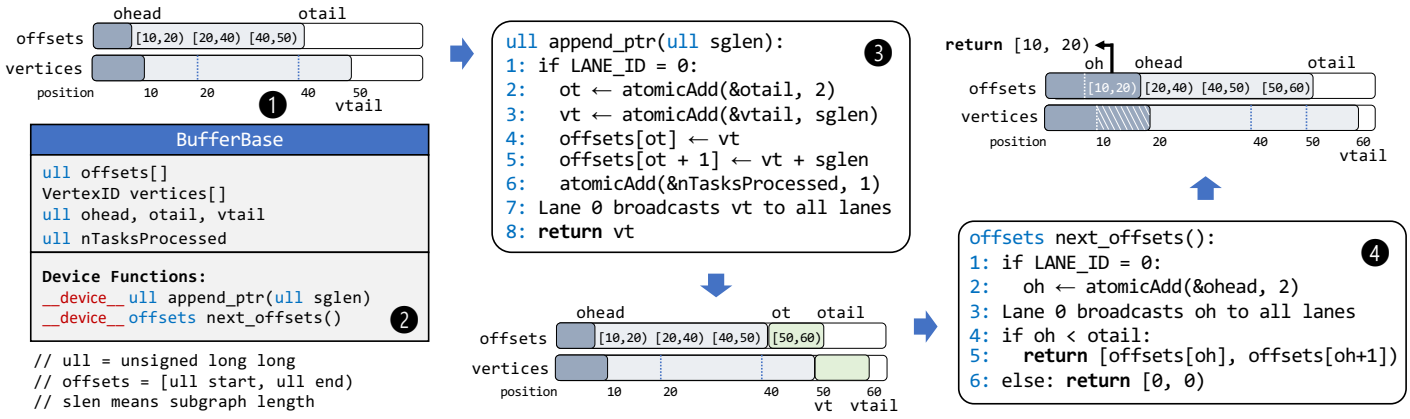


Fig. 13. BufferBase and Its Operations

We now explain the control loop of Figure 12(b). At Line 2, the GPU worker first checks whether the current level $L_i = \mathcal{B}[p_{rd}, p_{wr})$ is non-empty. If so, it invokes the application UDFs in Lines 3–4 to process and extend the current level.

The runtime supports two UDF execution modes. When the optional `process(.)` UDF is provided, GPU warps independently acquire parent subgraphs from L_i and generate their candidate elements into an auxiliary device buffer. The runtime stops assigning new parents once either all parents have been examined or the per-round expansion threshold η has been reached. Upon return, p'_{rd} indicates the boundary between processed and unprocessed parents. The subsequent `extend(.)` invocation materializes child subgraphs from the generated candidates and appends them to L_{i+1}. Making candidates explicit allows work to be distributed at candidate granularity, improving load balance when different parents have highly skewed candidate counts.

Alternatively, an application may implement `extend(.)` alone. In this mode, each warp acquires a parent, generates its candidates on the fly, and directly materializes the corresponding child subgraphs. This avoids the intermediate candidate buffer, although work distribution is then performed at parent granularity and may be less balanced under highly skewed candidate counts. In both modes, the runtime monitors the expansion threshold η and stops assigning new parents once the threshold is reached. Because multiple warps execute concurrently, warps already processing assigned parents may still append their remaining children, so the actual number of generated subgraphs may slightly exceed η.

After the UDF invocation completes, Line 5 records any unprocessed parents in L_i. Specifically, if $p'_{rd} < p_{wr}$, the GPU worker pushes $\langle p'_{rd}, p_{wr} \rangle$ onto the backtracking stack S_L. This pair compactly represents the deferred portion of the current level without copying its subgraphs. The worker then advances to L_{i+1} in Line 6 by setting $p_{rd} \leftarrow p_{wr}$ and $p_{wr} \leftarrow p'_{wr}$. The next iteration therefore continues with the newly generated frontier. For example, Figure 12(c) shows that after extending C and D, the remaining range $[4, 8)$ of L_2 is pushed onto S_L before execution proceeds to L_3.

When the current level becomes empty, the worker checks S_L in Line 7. If deferred work exists, Line 8 pops the most recently suspended range and restores it as the current level. The output boundary is reset in Line 9 so that this range can generate a fresh next-level segment. Figure 12(d) illustrates this process: after E and F are extended, the remaining range $[6, 8)$ is deferred and later restored to resume extension of G and H.

Finally, if both the current level and S_L are empty, all subgraphs reachable from the current input chunk have been processed. The worker therefore resets $\mathcal{B}$ (Line 11) and terminates the backtracking loop (Line 12).

Appendix C
Subgraph Buffer Implementation

The device buffer $\mathcal{B}$ and host spill stack $\mathcal{H}$ store variable-length subgraphs, each represented by a sequence of vertex IDs. To support compact storage and efficient movement of subgraphs between device and host, G-ThinkerCG uses a common data structure, `BufferBase`, whose device-side layout and operations are illustrated in Figure 13. Using the same representation for $\mathcal{B}$ and $\mathcal{H}$ allows the GPU worker to transfer batches of subgraphs between them without application-specific serialization or conversion.

Buffer Representation. As illustrated by ❶ and ❷ in Figure 13, a `BufferBase` consists of two preallocated arrays: `vertices`, which stores the vertex IDs of subgraphs contiguously, and `offsets`, which stores a pair `[start,end)` for each subgraph to identify its segment in `vertices`. This separation packs variable-length subgraphs compactly while keeping their descriptors fixed in size.

Three indices maintain the buffer state: `ohead` points to the next subgraph to be acquired for processing, `otail` marks the end of the subgraph descriptors in `offsets`, and `vtail` marks the end of the used region in `vertices`.

The buffer also maintains a counter `nTasksProcessed`, which tracks the number of subgraphs appended during the current expansion and is used to enforce the per-level threshold η.

Appending Subgraphs. Newly generated subgraphs are appended at warp granularity using `append_ptr(sglen)`, illustrated by ❸ in Figure 13. The first lane of a warp atomically advances `otail` and `vtail` to reserve one descriptor and `sglen` consecutive entries in `vertices`, respectively (Lines 2–3), and records the resulting `[start,end)` range in `offsets` (Lines 4–5). It also increments `nTasksProcessed` (Line 6), allowing the runtime and UDFs to detect when the expansion threshold η has been reached.

The reserved start position is then broadcast to all lanes (Line 7), after which the threads of the warp can cooperatively copy the subgraph's vertex IDs into the reserved segment of `vertices` in parallel. Since space reservation is performed atomically, multiple warps can concurrently append variable-length subgraphs without overlapping their storage.

Fetching Subgraphs. Warps acquire subgraphs for extension using `next_offsets()`, illustrated by ❹ in Figure 13. The first lane atomically advances `ohead` to claim the next descriptor (Line 2) and broadcasts the claimed position to all lanes (Line 3). If the position precedes `otail`, the primitive returns the corresponding `[start,end)` range (Lines 4–5), after which the warp cooperatively reads and processes the subgraph stored in `vertices`; otherwise, it returns an empty range (Line 6), indicating that no unclaimed subgraphs remain in the buffer. Since each descriptor position is claimed through an atomic operation, each subgraph is acquired by at most one warp even when many warps request work concurrently.

Together, `append_ptr()` and `next_offsets()` provide the device-side buffer interface exposed to application UDFs: user-written UDFs invoke these primitives to append newly generated subgraphs and fetch subgraphs for processing, while concurrent buffer bookkeeping is handled transparently by the runtime.

Interaction with Host Buffering. The host spill stack $\mathcal{H}$ uses the same subgraph representation. As described in Section III-B, when device space becomes insufficient, generated subgraphs are deferred to $\mathcal{H}$ for later processing. Once the current input chunk is exhausted, the GPU worker retrieves up to η most recently spilled subgraphs from the tail of $\mathcal{H}$ and transfers them into $\mathcal{B}$ as the next input batch. The common `offsets` + `vertices` representation enables such transfers without application-specific conversion, after which the restored subgraphs can be processed normally through `next_offsets()`.

Appendix D
GPU Worker Design and Interface

This appendix details the implementation and control flow of the GPU worker summarized in Section IV-B. The GPU worker is a dedicated host-side CPU thread that launches GPU kernels and manages both the device execution buffer $\mathcal{B}$ and the host-resident task stack $\mathcal{H}$. It also serves as the boundary between the master-managed scheduling structures and GPU execution: tasks assigned by the master are delivered through the worker's local buffers, while the GPU worker exclusively manages $\mathcal{B}$ and $\mathcal{H}$.

Algorithm 4 Pseudocode of the GPU Worker

```
1:  while true do
2:      wait to be notified by master
3:      if global_end_flag = true then
4:          break
5:      if L_t ≠ ∅ then
6:          H ← moveTasksFromCPU(L_t)                          ▷ ⑤
7:      v_pos ← 0                                  ▷ next vertex in L_v
8:      while true do
9:          if H ≠ ∅ then
10:             Pop up to η tasks from H to Bwr (= L_1)        ▷ ⑥
11:             if |W| > τ_c then
12:                 S ← moveTasksToCPU(H)                      ▷ ⑧
13:         else if v_pos < |L_v| then
14:             Bwr, v_pos ←
                generateInitialTasks<<<.>>>(L_v, v_pos)        ▷ ⑦
15:         else
16:             break
17:         Brd ← Bwr (i.e., L_1); set Bwr to L_2
18:         run the hybrid BFS-DFS procedure in Figure 12(b) to
    extend Brd = L_1      ▷ clear the iterator for correct invocation
    next time
19:     L_v ← ∅
20:     append itself to W and notify the master               ▷ ⑩
```

Algorithm 4 summarizes its control flow, and Figure 14 illustrates the corresponding data movement.

Task Handoff and Buffer Ownership. When the master assigns subgraph-tasks from the global stack $\mathcal{S}$ to the GPU worker, it places them in the worker's local task buffer $\mathcal{L}_t$ (Figure 14-③; Algorithm 2, Line 9). The GPU worker then converts these CPU-side tasks into the GPU-side subgraph representation through `moveTasksFromCPU(.)` and places them in $\mathcal{H}$ (Figure 14-⑤; Algorithm 4, Line 6). Tasks in $\mathcal{H}$ are subsequently transferred in batches into the device execution buffer $\mathcal{B}$ for GPU processing (Figure 14-⑥). Thus, the master never directly accesses either $\mathcal{H}$ or $\mathcal{B}$; it communicates with the GPU worker only through $\mathcal{L}_v$ and $\mathcal{L}_t$. This ownership boundary allows the GPU worker to manage its host- and device-side execution state without fine-grained synchronization with the master.

The stack $\mathcal{H}$ serves two related purposes. First, it receives tasks assigned to the GPU from $\mathcal{S}$ through $\mathcal{L}_t$. Second, as described in Section III-B, it stores subgraphs spilled during GPU execution when $\mathcal{B}$ approaches capacity. The GPU worker processes tasks already in $\mathcal{H}$ before generating new tasks from $\mathcal{L}_v$, preserving the near-DFS scheduling order described in Section III-B and preventing pending work in $\mathcal{H}$ from accumulating.

GPUContext and Execution Buffers. Figure 15 shows the `GPUContext` maintained by the GPU worker. It encapsulates the GPU-side execution state, graph data, buffers, and application UDFs required for subgraph enumeration. Its central data structure is the device execution buffer $\mathcal{B}$ described in Section III, which is logically organized into consecutive enumeration levels. During an extension from L_i to L_{i+1}, two `BufferT` views are maintained over $\mathcal{B}$: `Brd` identifies

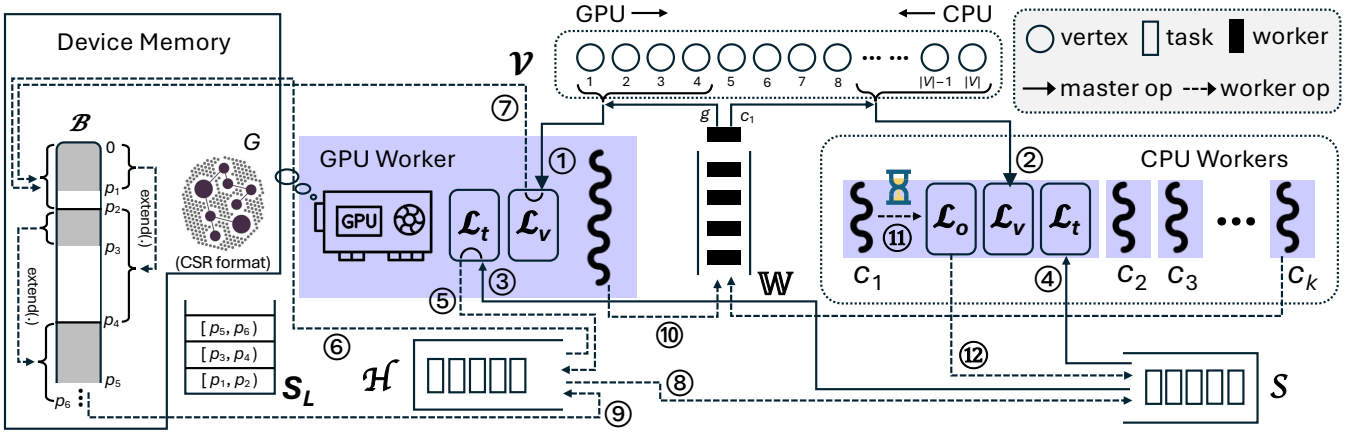


Fig. 14. System Architecture and Task Flows in G-ThinkerCG

```
GPUContext<BufferT, TaskT>
Internal Field:
BufferT B
External Fields:
BufferT H, Brd, Bwr
ull row_ptrs[], VertexID cols[]  // Graph in CSR format
VertexID sources[]  // L_v
B
offsets
vertices
ohead
otail
vtail
Brd  Bwr  are iterators over B
External Functions:
__host__ __device__ bool isOverflow(), __device__ bool isLevelFilled()
Device UDFs:
__device__ virtual void generateInitialTasks(VertexID* sources, ull* sources_num, ull* v_pos, BufferT& Bwr,
ull* row_ptrs, VertexID* cols)=0
__device__ virtual void process(BufferT& Brd, ull* row_ptrs, VertexID* cols) {}
__device__ virtual void extend(BufferT& Brd, BufferT& Bwr, BufferT& H, ull* row_ptrs, VertexID* cols)=0
Host UDFs:
virtual void moveTasksFromCPU(vector<TaskT*>& tasks, BufferT& H) = 0  // tasks is L_t
virtual void moveTasksToCPU(BufferT& H) = 0  // can call addTask(t) to add task t to S (CPU-side stack)
virtual void initialize() = 0, virtual void initLevel() {}
```

Fig. 15. The GPUContext class with UDFs

the current level L_i, from which UDF kernels acquire parent subgraphs using `BufferT::next_offsets()`, whereas `Bwr` identifies L_{i+1}, into which newly generated subgraphs are appended using `BufferT::append_ptr()`. These primitives are detailed in Appendix C.

The `GPUContext` also maintains $\mathcal{H}$ using the same `BufferT` representation. The storage of $\mathcal{H}$ is allocated in CUDA managed memory so that overflowing GPU kernels can directly append subgraphs to it, while the GPU worker can subsequently manipulate the same buffer from the host. The array `sources[]` provides the GPU-side view of the current vertex chunk in $\mathcal{L}_v$ and is consumed by `generateInitialTasks(.)` when creating the first enumeration level.

Initial-Task Generation. A GPU activation may start from either subgraph-tasks already present in $\mathcal{H}$ or initial vertices in $\mathcal{L}_v$. The former are given priority. If $\mathcal{H}$ is non-empty, the GPU worker transfers up to η tasks from its tail into `Bwr`, which becomes the first level L_1 of the next GPU execution batch (Algorithm 4, Lines 9–10). Otherwise, if unprocessed vertices remain in $\mathcal{L}_v$, the worker invokes `generateInitialTasks(.)` to construct L_1 directly from those vertices (Lines 13–14).

Since a GPU vertex chunk may contain substantially more than η vertices, `generateInitialTasks(.)` need not consume the entire $\mathcal{L}_v$ in one invocation. The variable v_{pos} records the next unprocessed vertex and is carried across successive invocations. Each invocation generates a bounded batch of initial subgraphs and advances v_{pos} accordingly (Line 14). Once L_1 has been constructed, the worker sets `Brd` to this level, initializes `Bwr` for L_2, and invokes the hybrid BFS-DFS extension procedure described in Section III and detailed in Appendix B (Lines 17–18). After that input batch is completely explored, the worker returns to $\mathcal{H}$ or $\mathcal{L}_v$ to construct the next batch.

Device UDFs. `GPUContext` exposes three application-defined device functions that specify GPU-side subgraph generation and extension:

- **`generateInitialTasks(.)`** creates initial subgraphs in `Bwr` ($= L_1$) from the vertices in $\mathcal{L}_v$ (Algo-

rithm 4, Line 14; Figure 14-⑦). The parameter `v_pos` identifies the next unprocessed source vertex, allowing a large vertex chunk to be consumed over multiple bounded GPU batches.

- **`process(.)`** is optional and generates candidate elements for parent subgraphs acquired from `Brd`. Applications that use this two-stage execution mode store the candidates in an auxiliary device buffer $\mathcal{Q}$, which is allocated by the host UDF `initialize()` and reset by `initLevel()` before processing a new level. Separating candidate generation from subgraph materialization allows candidate-level work distribution when different parent subgraphs have highly skewed numbers of candidate elements for extension.
- **`extend(.)`** materializes child subgraphs and appends them to `Bwr`. When `process(.)` is provided, `extend(.)` consumes the candidates previously generated in $\mathcal{Q}$; otherwise, it generates candidates and materializes their extensions directly. If an output subgraph cannot be accommodated in the device buffer, it is redirected to $\mathcal{H}$ according to the host-buffering mechanism of Section III-B.

These UDFs specify only application-dependent subgraph construction and pruning logic. Acquisition of parent subgraphs, reservation of output space, level transitions, backtracking, and overflow handling are provided by the runtime through the mechanisms described in Section III and Appendices B–C.

Expansion and Overflow Checks. Two runtime predicates coordinate bounded expansion with device-buffer capacity. Both predicates are exposed to application UDFs, allowing user-defined GPU code to query the runtime state during subgraph extension.

The device-side predicate `isLevelFilled(.)` indicates whether the current expansion should stop acquiring new parent subgraphs. It becomes true when either the per-round expansion threshold η has been reached, as tracked by `Bwr.nTasksProcessed`, or the device buffer is approaching capacity. As discussed in Section III, η is a soft expansion bound because warps that have already acquired parents may finish generating their remaining children after the threshold is reached.

The predicate `isOverflow(.)` monitors the physical capacity of $\mathcal{B}$. Since `BufferT` separately stores subgraph descriptors in `offsets[]` and variable-length contents in `vertices[]`, exhaustion of either array constitutes an impending overflow. We conservatively regard the buffer as full when the utilization of either array exceeds 95%. This margin accommodates concurrent reservations by active warps while allowing most of the allocated device memory to remain usable.

During `extend(.)`, a child that cannot be safely appended to $\mathcal{B}$ is redirected to $\mathcal{H}$ rather than discarded. After the kernel returns, the GPU worker evaluates `isOverflow(.)` on the host. If overflow has been triggered, the portion of the output level already materialized in `Bwr` is also deferred to $\mathcal{H}$, freeing the device buffer so that the worker can backtrack and continue processing other resident work. This is the implementation-level realization of the host-buffering procedure illustrated in Figure 3.

CPU-GPU Task Conversion. The GPU and CPU implementations of an application may use different physical representations for the same logical subgraph-task. `GPUContext` therefore provides two host UDFs for crossing this representation boundary. The GPU worker invokes `moveTasksFromCPU(.)` when tasks assigned through $\mathcal{L}_t$ must be converted from `TaskT` objects into the `BufferT` representation used by $\mathcal{H}$. Conversely, `moveTasksToCPU(.)` converts tasks selected from $\mathcal{H}$ into CPU-side `TaskT` objects and inserts them into the global stack $\mathcal{S}$. These conversions are performed by the GPU worker rather than the master, preserving the ownership boundary around $\mathcal{H}$ and $\mathcal{B}$.

The second direction is activated only when CPU capacity is available. Specifically, when the number of idle CPU workers exceeds τ_c, the GPU worker may convert a batch of tasks from $\mathcal{H}$ and return them to $\mathcal{S}$ (Algorithm 4, Line 12; Figure 14-⑧). The master can subsequently assign these tasks to idle CPU workers. Together, $\mathcal{L}_t \rightarrow \mathcal{H}$ and $\mathcal{H} \rightarrow \mathcal{S}$ implement the two representation-conversion paths required by the bidirectional task redistribution described in Section IV-A.

GPU Worker Execution Flow. We finally summarize Algorithm 4. After being awakened by the master (Line 2), the GPU worker first checks the global termination flag (Lines 3–4) and imports any tasks assigned through $\mathcal{L}_t$ into $\mathcal{H}$ (Lines 5–6). It then repeatedly constructs an input batch, giving priority to pending tasks in $\mathcal{H}$ over new vertices in $\mathcal{L}_v$ (Lines 9–14). Each input batch is fully explored using the hybrid BFS-DFS procedure before the next batch is selected (Lines 17–18). During this process, newly generated work may be spilled back to $\mathcal{H}$, and surplus tasks in $\mathcal{H}$ may be returned to $\mathcal{S}$ when CPU workers are underutilized (Lines 11–12).

The activation ends only when both $\mathcal{H}$ and the assigned vertex chunk $\mathcal{L}_v$ have been exhausted (Lines 15–16). The worker then clears $\mathcal{L}_v$, inserts itself into the idle-worker queue $\mathbb{W}$, and notifies the master (Lines 15–16). It remains idle until the master assigns another batch or sets the global termination flag (Lines 2–4).

APPENDIX E
DATA SOURCES

Table IV summarizes the graph datasets used in our evaluation and their sources. MCE and SM are evaluated on the larger graph datasets listed in the first group. MQC currently requires maintaining a two-hop CSR representation on the GPU by following the implementation of cuQC [41], which incurs a substantially larger memory footprint and prevents these large graphs from fitting in GPU device memory. We therefore evaluate MQC using the smaller graph datasets listed in the second group. All datasets are obtained from

TABLE IV
GRAPH DATASETS AND THEIR SOURCES

Graph	Data Source
Graphs for MCE and SM:	
soc-sinaweibo	https://networkrepository.com/soc-sinaweibo.php
web-wikipedia_link_en	https://networkrepository.com/web-wikipedia-link-en13-all.php
edit-frwiki	http://konect.cc/networks/edit-frwiki/
edit-cebwiki	http://konect.cc/networks/edit-cebwiki/
edit-svwiki	http://konect.cc/networks/edit-svwiki/
edit-eswiki	http://konect.cc/networks/edit-eswiki/
edit-dewiki	http://konect.cc/networks/edit-dewiki/
edit-enwiktionary	http://konect.cc/networks/edit-enwiktionary/
wikipedia_link_sh	http://konect.cc/networks/wikipedia_link_sh/
wikipedia_link_de	http://konect.cc/networks/wikipedia_link_de/
edit-frwiktionary	http://konect.cc/networks/edit-frwiktionary/
wikipedia_link_ru	http://konect.cc/networks/wikipedia_link_ru/
wikipedia_link_fr	http://konect.cc/networks/wikipedia_link_fr/
wikipedia_link_sr	http://konect.cc/networks/wikipedia_link_sr/
orkut-links	http://konect.cc/networks/orkut-links/
edit-arwiki	http://konect.cc/networks/edit-arwiki/
wikipedia_link_nl	http://konect.cc/networks/wikipedia_link_nl/
wiki-Talk	http://konect.cc/networks/wiki-Talk/
wikipedia_link_it	http://konect.cc/networks/wikipedia_link_it/
wiki-topcats	https://snap.stanford.edu/data/wiki-topcats.html
Graphs for MQC:	
socfb-B-anon	https://networkrepository.com/socfb-B-anon.php
Flixster	https://networkrepository.com/soc-flixster.php
soc-pokec	http://konect.cc/networks/soc-pokec-relationships/
Hyves	http://konect.cc/networks/hyves/
Douban	http://konect.cc/networks/douban/
WordNet	http://konect.cc/networks/wordnet-words/
slashdot-threads	http://konect.cc/networks/slashdot-threads/
FB-Pages	https://networkrepository.com/fb-pages-politician.php
edit-kywiktionary	http://konect.cc/networks/edit-kywiktionary/
wikilens-ratings	http://konect.cc/networks/wikilens-ratings/
edit-sswiki	http://konect.cc/networks/edit-sswiki/

TABLE V
EFFECT OF SYSTEM PARAMETERS

Parameter	Value	wikipedia_link_fr	edit-eswiki
η / #{warps}	1000	230.3	39.5
	2000	**199.8**	**28.2**
	5000	277.1	30.8
	10,000	399.0	32.3
CPU Chunk	10	208.4	34.9
	100	162.9	28.0
	200	**160.5**	**19.3**
	500	210.8	26.1
GPU Chunk	10K	187.3	26.3
	100K	**160.5**	**19.3**
	500K	194.0	27.0
τ_c	1	180.2	20.1
	n_c / 2	**160.5**	**19.3**
	n_c / 4	203.3	19.5

publicly available graph repositories, including the Network Repository, KONECT, and SNAP.

APPENDIX F
SYSTEM PARAMETER STUDY

Without loss of generality, we demonstrate the effect of key system parameters on CPU+GPU execution using the MCE application with two representative datasets, `wikipedia_link_fr` and `edit-eswiki`. Table V varies one parameter at a time while keeping the others at their default values. Overall, the default setting provides robust performance across both datasets.

TABLE VI
EFFECT OF CPU CHUNK SIZE ON SM PERFORMANCE

G	Query	1	10	50	100	200
edit-cebwiki	P1	**1.1**	2.8	7.2	6.8	15.7
	P2	**4.4**	5.4	8.7	15.7	21.9
	P3	**10.8**	11.2	11.3	20.7	34.9
	P4	**19.3**	21.2	86.9	22.2	67.7
	P5	**2.4**	4.3	10.2	14.5	7.6
orkut-links	P1	**4.1**	7.7	15.0	18.3	29.4
	P2	**52.7**	54.5	58.3	63.7	66.3
	P3	**168.3**	173.9	181.2	184.6	187.4
	P4	**171.5**	176.9	188.0	185.7	199.6
	P5	**20.6**	22.1	24.9	32.1	43.5
edit-arwiki	P1	**8.3**	12.7	21.4	26.9	29.0
	P2	**129.2**	129.7	133.5	134.8	131.6
	P3	**466.9**	477.1	483.5	496.2	512.3
	P4	**443.7**	476.9	528.7	560.4	728.9
	P5	**7.5**	12.3	23.4	37.9	46.0
wiki-topcats	P1	**1.3**	5.2	9.8	13.6	20.3
	P2	**5.6**	7.7	13.0	22.5	29.5
	P3	**14.2**	15.2	21.9	27.4	32.5
	P4	**17.6**	18.8	22.1	31.6	33.9
	P5	**2.0**	3.2	10.0	11.2	18.1

For the GPU expansion threshold η, a small value limits the parallel work exposed to GPU warps, whereas an excessively large value allows larger frontiers to accumulate before backtracking, increasing memory pressure and potentially triggering more frequent host spilling. The default $\eta = 2000 \cdot \#\{\text{warps}\}$ achieves the best performance on both datasets.

The CPU and GPU chunk sizes reflect a different tradeoff in heterogeneous scheduling. A small CPU chunk allows CPU workers to return promptly for dynamically exposed work, but increases scheduling and waiting overhead, whereas an overly large chunk can tie a worker to too much initial work. In contrast, the GPU requires sufficiently large chunks to expose abundant parallelism and amortize kernel-launch and scheduling overhead; overly large chunks can reduce the flexibility of load redistribution. We therefore use a CPU chunk size of 200 and a GPU chunk size of 100K.

The GPU-to-CPU task-exchange threshold τ_c controls how aggressively GPU work is returned for CPU processing. A small threshold can trigger unnecessary task migration when only a few CPU workers become idle, whereas a large threshold may leave available CPU capacity underutilized. Setting $\tau_c = n_c/2$ provides the best overall performance in our study. Finally, we transfer η subgraph-tasks per CPU-to-GPU steal, providing one full GPU chunk of work, and 5000 subgraph-tasks per GPU-to-CPU steal to provide sufficient work even when the transferred tasks have shallow search trees and to reduce frequent repeated steals.

APPENDIX G
CPU CHUNK SIZE TUNING FOR SM

Table VI reports the CPU+GPU running time of SM with different CPU chunk sizes, while keeping the other system

TABLE VII
EFFECT OF THE GPU CHUNK SPLITTING FACTOR k ON MQC PERFORMANCE

G	$\|V\|$	$\|E\|$	d_{max}	τ_{size}	γ	# root tasks	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=10$
socfb-B-anon	2.938M	41.92M	4356	20	0.95	399,864	801.2	**607.6**	678.6	629.5	656.3	683.5
Flixster	2.523M	15.84M	1474	30	0.95	41,319	117.0	25.4	**25.3**	27.1	27.7	30.1
soc-pokec	1.633M	44.60M	14,854	20	0.90	573,021	239.2	**210.0**	241.4	225.6	223.7	231.7
Hyves	1.403M	5.555M	31,883	20	0.80	3785	39.2	**24.4**	26.6	27.3	27.1	26.8
Douban	154.9K	654.3K	287	8	0.50	20,181	26.4	**6.2**	6.8	8.0	9.8	12.7
WordNet	146.0K	1.314M	1008	5	0.80	94,500	49.3	7.5	7.2	**7.0**	9.7	11.0
slashdot-threads	51.08K	233.1K	2915	6	0.70	8421	28.8	**1.5**	1.8	1.8	3.5	3.6
FB-Pages	50.52K	1.638M	1469	23	0.90	16,009	406.3	250.1	256.2	250.2	**239.0**	249.0
edit-kywiktionary	39.88K	152.9K	23,237	7	0.60	5294	40.6	6.7	6.7	**4.6**	5.4	12.4
wikilens-ratings	5111	53.70K	1730	10	0.72	1262	38.3	**31.7**	32.0	32.2	33.4	33.4
edit-sswiki	2022	22.23K	604	10	0.70	490	112.3	**96.9**	104.3	104.6	105.8	106.0

parameters as the default values. A CPU chunk size of 1 consistently provides the best performance across the evaluated workloads. As discussed in Section VII, GPU execution is substantially faster than CPU execution for SM, while its uniformly shallow enumeration trees provide little opportunity for CPUs to handle deep stragglers. Increasing the CPU chunk size therefore assigns more work to the slower CPU path and delays overall completion. We accordingly use a CPU chunk size of 1 for all SM experiments.

APPENDIX H
GPU CHUNK SIZE SELECTION FOR MQC

Table VII evaluates the splitting factor k in our adaptive GPU chunk size $\max(|\mathcal{V}|/k, 1000)$ for MQC. With $k = 1$, the GPU can take all root tasks in a single chunk, leaving little opportunity for CPU workers to process potentially deep tasks from the back of $\mathcal{V}$. Setting $k = 2$ substantially improves performance by allowing the GPU and CPU workers to start processing complementary portions of the workload concurrently. Further increasing k provides no consistent benefit and can reduce GPU efficiency due to smaller chunks. We, therefore, use $k = 2$ as the default for MQC.